

Tehnici de programare - TP



Cursul 14 – Recapitulare

Ș.l. dr. ing. Cătălin Iapă

catalin.iapa@cs.upt.ro

Decimal System: $1+1=2$

Binary System: $1+1=10$

Boolean Algebra: $1+1=1$

Non-Programmers:



Analiza Algoritmilor

O notație des întâlnită pentru performanța unui algoritm este $O(expr)$, unde $expr$ este o funcție, în general exprimată în funcție de variabila n . Notația $O(...)$ (en: **big O notation**) reprezintă **numărul de operații efectuate în cazul cel mai defavorabil**, în funcție de numărul n de elemente de intrare.

Omega (Ω) notation este folosită pentru a descrie **limita inferioară** a timpului de execuție sau spațiului de memorie al unui algoritm.

Theta (Θ) notation combină atât notația Big O (limita superioară) cât și notația Omega (limita inferioară).

Se folosește pentru a indica **o limită strânsă asupra performanței algoritmilor**, arătând că timpul de execuție sau spațiul de memorie este în mod aproximativ **proporțional** cu funcția specificată.

Masurarea timpului de executie

```
#include <time.h>
```

```
clock_t start, end;
```

```
double cpu_time_used;
```

```
start = clock();
```

```
... /* Do the work. */
```

```
end = clock();
```

```
cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
```

Generarea de numere aleatoare

*// genereaza o secventa de numrere pseudoaleatoare,
distincta la fiecare apel*

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

int main(void)

{

// Foloseste ora curenta pentru initializarea PRNG-ului

srand(time(0));

for(int i = 0; i<4; i++){

printf(" %d ", rand());

}

return 0;

}

Recursivitate

În C, **recursivitatea** se realizează prin intermediul funcțiilor, care se pot *autoapela*.

O funcție trebuie **definită** iar apoi se poate apela.

Recursivitatea constă în faptul că **în definiția unei funcții apare apelul ei însăși**. Acest apel, care apare în însăși definiția funcției, se numește **autoapel**.

Primul apel, făcut în altă funcție, se numește **apel principal**.

Recursivitate

Funcția factorial
implementată iterativ
(**nerecursiv**)

```
int fact(int n){  
    int p = 1, i;  
    for(i = 1 ; i <= n ; i ++)  
        p = p * i;  
    return p;  
}
```

Funcția factorial
implementată **recursiv**

```
int fact(int n){  
    if(n == 0)  
        return 1;  
    else  
        return n * fact(n-1);  
}
```

Greedy

Algoritmii de tip greedy se caracterizează prin **luarea unor decizii rapide care duc la găsirea unei soluții a problemei.**

Nu întotdeauna asemenea decizii rapide conduc la o soluție optimă, dar vom vedea că există anumite tipuri de probleme unde **se pot obține soluții optime sau foarte apropiate de optim.**

Există o clasă de probleme pentru care metoda Greedy conduce la rezultate optime, și anume atunci când ***soluția optimă globală se compune întotdeauna din soluțiile optime locale***, de la fiecare pas.

Divide et Impera - Principii

Este o **metodă fundamentală de proiectare a algoritmilor** care poate să conducă la soluții deosebit de eficiente.

Principiul de bază al acestei tehnici este acela de a **descompune în mod repetat o problemă complexă în două sau mai multe subprobleme de același tip**, urmată de combinarea soluțiilor acestor subprobleme pentru a obține soluția problemei inițiale.

Întrucât subproblemele rezultate din descompunere sunt de același tip cu problema inițială, **metoda se exprimă în mod natural printr-o funcție recursivă**.

Apelul recursiv se continuă până **în momentul în care subproblemele devin banale și soluțiile lor evidente**.

Metoda Backtracking - principii

Scopul algoritmului concret poate să fie determinarea unei soluții rezultat sau a tuturor soluțiilor rezultat, fie **în scopul afișării lor**, fie pentru **a alege una optimă** din punctul de vedere al unor criterii de optimizare (minimizare sau maximizare).

O metodă simplă de selectare a soluțiilor rezultat este aceea **de a genera toate soluțiile posibile și de a verifica satisfacerea condițiilor interne** (*căutare exhaustivă* în întregul spațiu al soluțiilor posibile). Această metodă necesită însă un **timp de execuție foarte mare**.

Metoda Backtracking - algoritm

Mai simplu spus, algoritmi de tip backtracking funcționează în felul următor:

- soluția problemei **se construiește succesiv**, pas cu pas
- dacă la un pas există mai multe posibilități de continuare, **se vor încerca pe rând fiecare dintre ele**
- dacă s-a ajuns într-un punct în care nu se mai poate continua, **se revine la pasul anterior** pentru a se încerca următoarea variantă posibilă
- după ce s-au epuizat toate posibilitățile de la pasul anterior, **se revine cu încă un pas mai înainte** și tot așa, în mod recursiv, până când au fost încercate toate posibilitățile din toți pașii

Metoda Backtracking – implementare

O implementare recursivă simplă a algoritmului backtracking în limbajul de programare C:

```
void back(int k){  
    for(int i=0;i<n;i++)  
    {  
        v[k]=i;  
        if (valid(k))  
            if(solutie(k))  
                afisare();  
            else  
                back(k+1);  
    }  
}
```

Metoda Backtracking – exerciții

Exercițiul 1:

Se primește un cuvânt din lina de comandă. Să se afișeze toate anagramele sale.(DEX anagramă: schimbare a ordinii literelor unui cuvânt, pentru a obține alt cuvânt; cuvânt obținut prin această schimbare.)

Metoda Backtracking – exerciții

```
int main(int argc, char* argv[])  
{  
char *cuv=strdup(argv[1]);  
int n=strlen(cuv);  
back(1, n, cuv);  
return 0;  
}
```

Metoda Backtracking – exerciții

```
void back(int k, int n, char* cuv)
{
    for (int i=1; i<=n; i++){
        st[k]=i;
        if (valid(st, k)){
            if (solutie(st, k, n)){
                afisare(st, k, cuv);    }
            else{
                back(k+1, n, cuv);    }
            }
        }
    }
```

Metoda Backtracking – exerciții

```
int valid(int st[], int k){  
    for (int i=1; i<k; i++){  
        if (st[i]==st[k]){  
            return 0;    }  
        }  
    return 1;}
```

```
int solutie(int st[], int k, int n){  
    return (k==n);}
```

```
void afisare(int st[], int k, char *cuv){  
    for (int i=1; i<=k; i++){  
        printf("%c", cuv[st[i]-1]);    }  
    printf("\n");}
```


Metoda Backtracking – exerciții

Exercițiul 2:

*Să se genereze toate șirurile de cifre distincte a
căror sumă este egală cu n citit de la tastatură.*

Metoda Backtracking – exerciții

```
int main(){  
scanf("%d", &x);  
back(0);  
return 0;  
}
```

Metoda Backtracking – exerciții

```
void back(int k){  
    for (int i = 1; i < n; i++) {  
        v[k] = i;  
        if (valid(k)) {  
            if (solutie(k)) {  
                afisare(k);  
            }  
            else {  
                back(k + 1);  
            }  
        }  
    }  
}
```

Metoda Backtracking – exerciții

```
int valid(int k){  
    int suma = 0;  
    for (int i = 0; i < k; i++)    {  
        if (v[i] == v[k])    {  
            return 0;    }  
        suma = suma + v[i];  
    }  
    suma = suma + v[k];  
    if (suma > x)    {  
        return 0;    }  
    else    {  
        return 1;    }  
}
```

Metoda Backtracking – exerciții

```
int solutie(int k){  
    int suma = 0;  
    for (int i = 0; i <= k; i++) {  
        suma = suma + v[i];    }  
    if (x == suma) {  
        return 1;    }  
    else {  
        return 0;    }  
}  
  
void afisare(int k){  
    for (int i = 0; i <= k; i++) {  
        printf("%d ", v[i]);    }  
    printf("\n");  
}
```

Structuri de date

Structurile de date fundamentale ale limbajului C (**tabloul, structura și uniunea**) sunt fixe din punct de vedere al formei și al dimensiunii. Aceste structuri de date se numesc **statice**.

În practica programării însă, există multe probleme care presupun structuri ale informației mult mai complicate (de ex.: **liste, arbori, grafuri**), a căror caracteristică principală este aceea că ***se modifică structural în timpul execuției programului***. Rezultând o modificare dinamică atât a formei cât și a dimensiunii lor, aceste structuri de date se numesc **dinamice**.

Liste liniare simplu înlănțuite

Listele liniare simplu înlănțuite sunt structuri de date care permit stocarea și gestionarea unui set de elemente într-o ordine specifică.

Fiecare element este legat printr-un pointer către **următorul element din listă**, astfel încât elementele pot fi accesate în **ordine secvențială**.

Lista simplu înlănțuită este diferită de alte structuri de date, cum ar fi vectorii sau matricele, deoarece lista **poate fi extinsă sau micșorată** în funcție de nevoile aplicației.

Liste simplu înlănțuite - nodul

// un element al listei – un nod

```
typedef struct elem{
```

```
    int info;
```

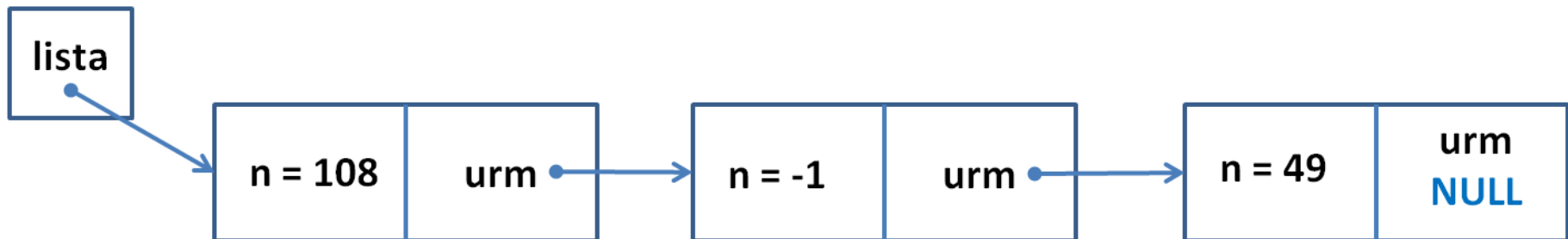
// informatia utila

```
    struct elem *urm;
```

// urmatorul nod

```
}elem;
```

```
elem *lista = NULL;
```



Adaugarea unui nod la început

2 metode similare

Transmitem *pointerul* spre inceputul listei ca parametru:

```
elem * adaugaInceput(elem *lista,  
int n)  
{  
    elem *nou;  
    nou=nod_nou(n,lista);  
    lista=nou;  
    return lista;  
}
```

La apelare:

```
lista = adaugaInceput(lista, 10);
```

Transmitem *adresa pointerului* spre inceputul listei ca parametru:

```
void adaugaInceput(elem **plista,  
int n)  
{  
    elem *nou;  
    nou=nod_nou(n,*plista);  
    *plista=nou;  
}
```

La apelare:

```
adaugaInceput(&lista, 10);
```

Liste ordonate

Listele ordonate se construiesc de la bun început ordonate, plecând de la faptul că lista vidă este în mod intrinsec ordonată, iar **nodurile ulterioare se inserează astfel încât să nu se strice ordinea deja existentă.**

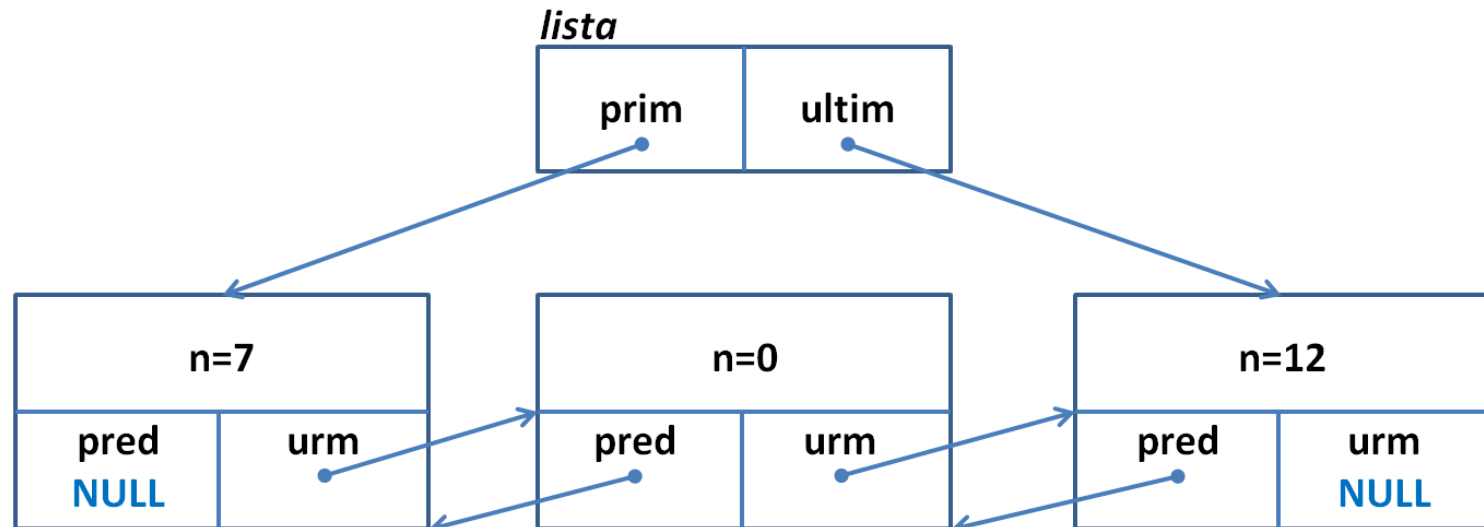
Într-o listă ordonată **crește performanța operațiilor de căutare.**

Membrul care servește la identificarea nodurilor și asupra căruia operează criteriul de ordonare este numit **membrul sau câmpul cheie.**

Liste dublu înlănțuite - Structura

O listă dublu înlănțuită păstrează pentru fiecare element atât **un pointer către următorul element din listă** (*urm* - următor), cât și **un pointer către elementul anterior** (*pred* - predecesor).

De asemenea, pe lângă pointerul care reține adresa primului element, este util să avem **un pointer care reține adresa ultimului element**.



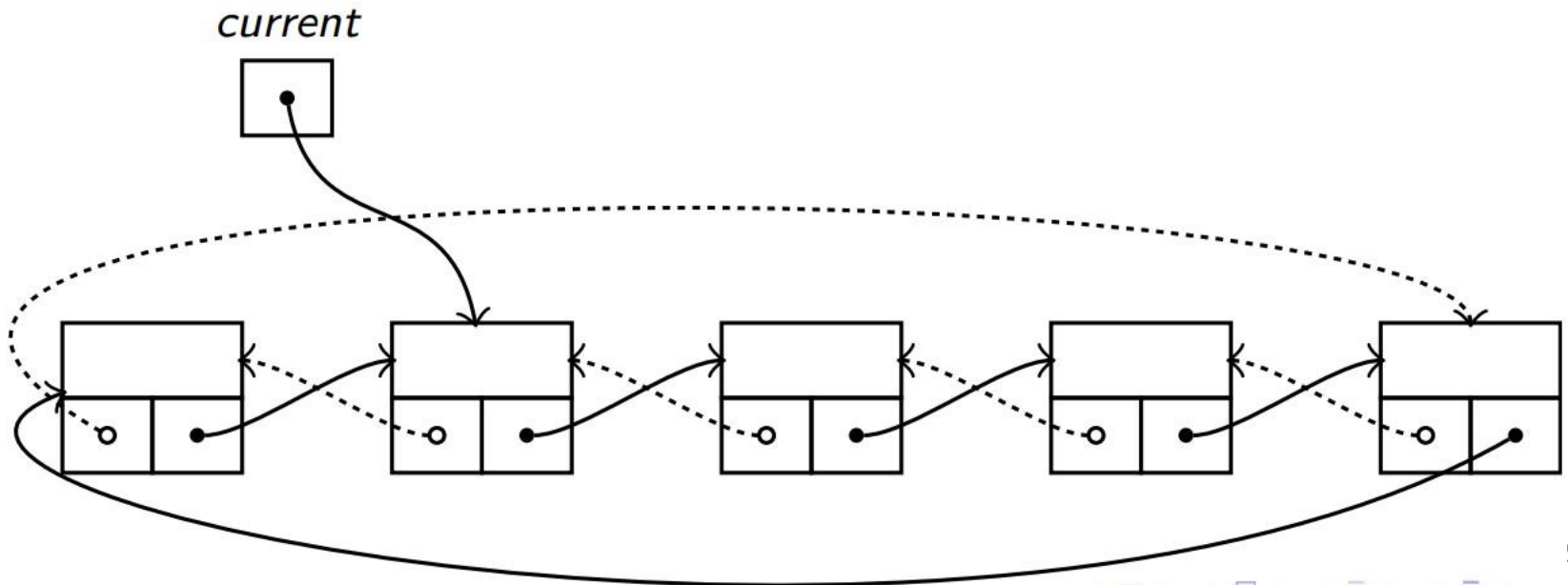
Liste simplu sau dublu înlănțuite?

Pentru **liste scurte**, de doar câteva elemente, timpul necesar pentru iterarea unei **liste simplu înlănțuite** este compensat de timpul mai mic necesar pentru alte operații, deoarece acestea sunt mai simple decât la listele dublu înlănțuite.

Dacă în schimb **lista trece de un anumit număr de elemente** și **operațiile de ștergere sau inserare înainte de elementul curent sunt frecvente**, atunci deja diferența între cele două tipuri de date devine semnificativă.

Liste circulare

Listele dublu înlanțuite se pot organiza ușor ca **liste circulare** (în care nu există nici un element prim sau ultim ci **doar un pointer curent spre orice nod al listei** și în care nu avem **NULL** la nici un **pointer pred sau urm al nodurilor**).



Exercițiu - liste

Aplicația 7.2: Să se scrie o funcție care primește o listă și returnează **lista respectivă cu elementele inversate**. Funcția va acționa doar asupra listei originare, fără a folosi vectori sau alocare de noi elemente.

Exercițiu - liste

```
elem *inversare(elem *lista){
if(lista == NULL)  {
return lista;  }
elem *prev, *q,*aux;
for(prev=lista, q=lista->urm;q->urm!= NULL; prev = q, q=aux)
{
    aux=q->urm;
    q->urm = prev;  }
q->urm=prev;
lista->urm=NULL;
lista=q;
return lista;
}
```

Argumente din linia de comandă

```
int main(int argc, char *argv[])
{
    int i, n;
    for (i = 0; i < argc; i++) {
        printf("Parametrul i este: %s", argv[i]);
        printf("\n");
    }
    return 0;
}
```

Dacă rulăm programul astfel:

./prg Ana 23 "Un program"

Va afișa:

Parametrul 0 este: ./prg

Parametrul 1 este: Ana

Parametrul 2 este: 23

Parametrul 3 este: Un program

Funcții cu nr variabil de argumente

Să se calculeze suma argumentelor variabile (întregi). Numărul argumentelor variabile va fi specificat de argumentul fix al funcției.

```
#include <stdarg.h>
```

```
int suma(int n, ...) {  
    va_list va;  
    int suma = 0;  
    va_start(va, n);  
  
    for(int i = 0; i < n; i++) {  
        suma += va_arg(va, int);  
    }  
    va_end(va);  
    return suma;  
}
```

La apel:

suma(3, 1, 2, 3) -> rez: 6

suma(5, 5, 5, 5, 5, 5) -> rez: 25

suma(1, 100) -> rez: 100

Funcții cu nr variabil de argumente



Cocktail-urile sunt ca funcțiile cu număr variabil de argumente: primesc număr variabil de ingrediente

Arizona Sunset

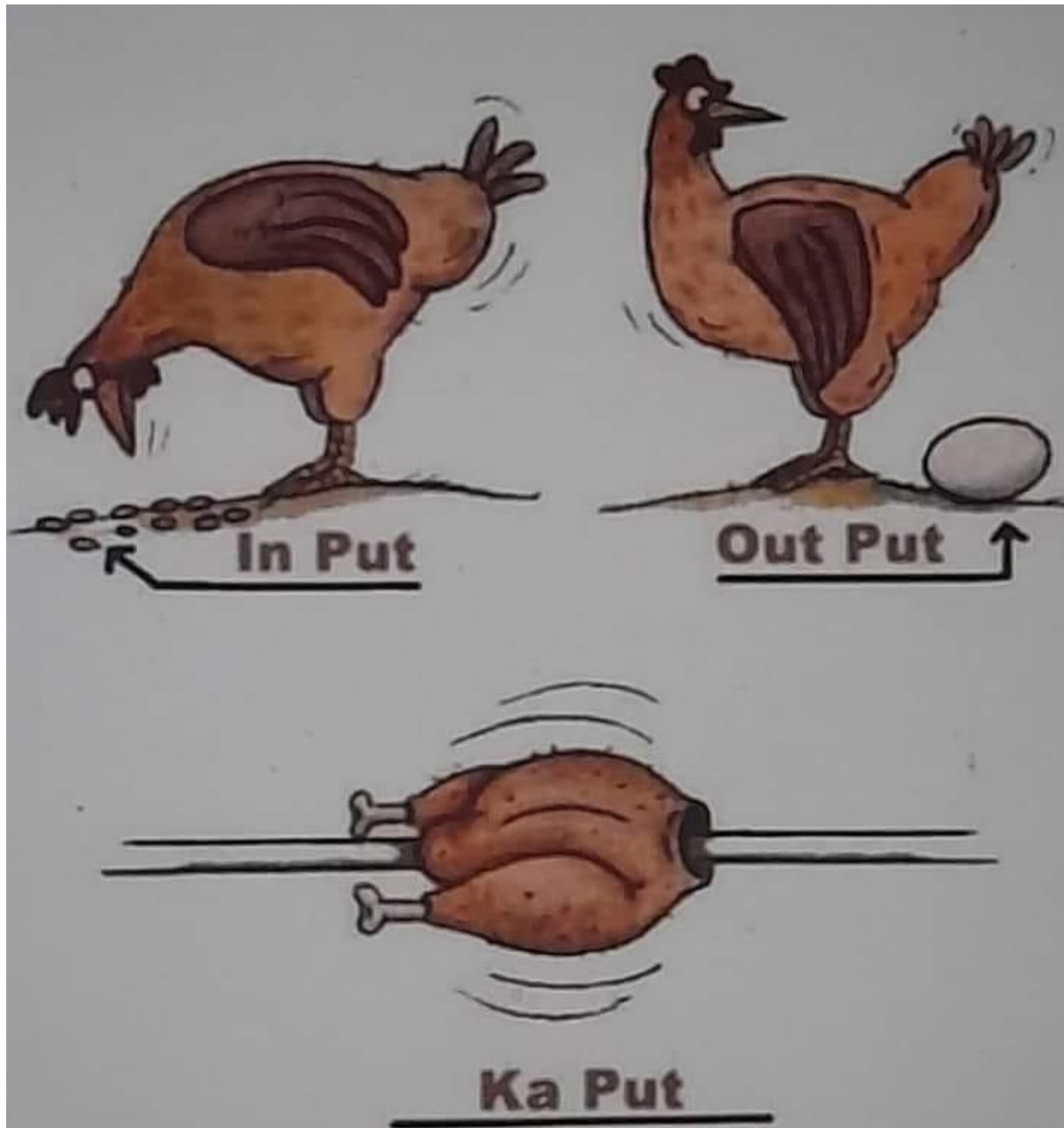
1. 500 ml Sprite
2. 500 ml suc de portocale
3. Sirop Grenadine
4. Cirese Maraschino sau felii de portocala pentru decor
5. Gheata macinata sau cuburi

Cinderella

1. 400 ml suc de portocale
2. 400 ml suc de ananas
3. 75 apa minerala
4. 75 ml suc de lamaie
5. 50 ml sirop de zahar
6. Gheata macinata
7. Feli de ananas sau de portocala pentru ornarea paharului

Tropical Breeze

1. 300 ml suc de portocale
2. 300 ml suc de mango
3. 300 ml suc de ananas
4. Gheata macinata



Scrierea într-un fișier text

Pentru scriere vom deschide fișierul în mod de scriere (**w** - write), sau de adăugare (**a** - append) iar apoi vom folosi funcții cum sunt **fprintf**, **fputs**, **fputc**:

```
FILE *f;
```

```
f = fopen("fis.txt", "w");
```

```
if( f == NULL) exit(1);
```

```
char sir[] = "Exemplu", c = 'A';
```

```
fprintf(f, "%s",sir);
```

```
fputs(sir, f);
```

```
fputc(c,f);
```

```
fclose(f);
```

Citirea din fișier text

```
FILE *g;
```

```
g = fopen("fis.txt", "r");
```

```
if( g == NULL) exit(1);
```

```
char sir[21] , c;
```

```
fscanf(g, "%20s", sir);           // fscanf citește cuvinte – citește  
text până când întâlnește un spațiu, un tab, un enter sau EOF
```

```
fgets(sir, 20, g);                 // fgets citește fraze – citește  
până când întâlnește enter sau EOF (sfârșit de fișier)
```

```
c = fgetc(g);
```

```
fclose(f);
```

Citirea din fișier text – `feof(f)`

Putem verifica dacă s-a ajuns la sfârșitul fișierului cu ajutorul funcției *`feof(f)`*.

Funcția *`feof(f)`* returnează `true` (diferit de 0), doar dacă s-a ajuns la sfârșit de fișier.

Funcția *`feof`* se poate folosi și la citirea din fișier, pentru a se testa dacă s-a ajuns la sfârșit (ex: *`while(!feof(fis)){...citire..}`*), dar se poate folosi pentru aceasta și direct valoarea returnată de funcțiile de citire.

Accesul la fișiere binare

//exemplu de program care copiază conținutul unui fișier în altul

```
#include <stdio.h>
int main() {
    FILE *in_file, *out_file;
    char buffer[1000];
    int bytes_read;
    in_file = fopen("input.bin", "rb");
    out_file = fopen("output.bin", "wb");
    while ((bytes_read = fread(buffer, 1, 1000, in_file)) > 0) {
        fwrite(buffer, 1, bytes_read, out_file);
    }
    fclose(in_file);
    fclose(out_file);
    return 0;
}
```

Accesul la fișiere

Pentru acces direct oriunde în fișier, nu doar secvențial vom folosi funcția **fseek**:

int fseek(FILE *fis, long offset, int reper)

-setează poziția curentă din fișier, folosind ca punct de referință reperul specificat.

Reperul poate fi: **SEEK_SET** - începutul fișierului, **SEEK_CUR** - poziția curentă, **SEEK_END** - sfârșitul fișierului.

fseek(fis, 0, SEEK_END) - mută poziția curentă la sfârșit de fișier.

fseek(fis, -5, SEEK_CUR) - mută poziția crt. cu 5 octeți în stânga.

long ftell(FILE *fis)

- returnează poziția curentă (offsetul) din fișier, față de începutul acestuia. La această poziție vor avea loc următoarele operații de citire sau de scriere.

WHEN YOU START PROGRAMMING



AFTER A WHILE



Structuri

```
struct Produs{  
    char nume[200];  
    float pret;  
    int stoc;  
};
```

```
struct Produs p1, p2={"ciocolata", 12, 100}, v[10], *p;  
p1=p2;  
p2.pret = 7.5;
```

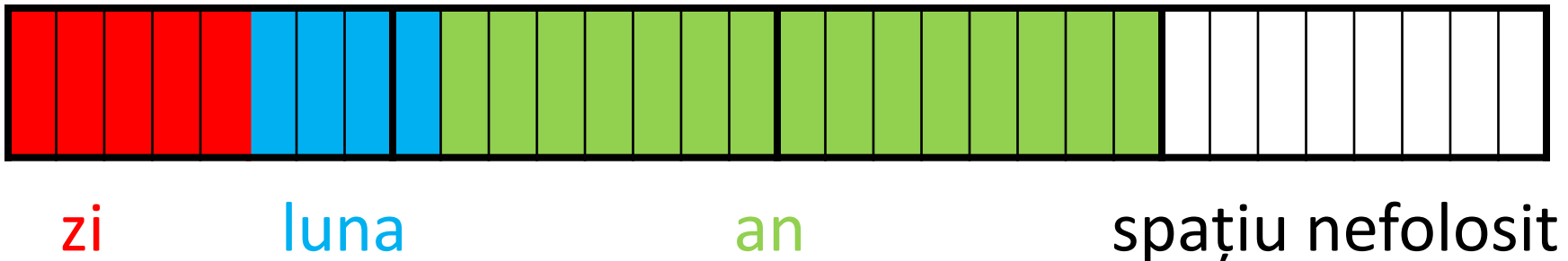
```
v[3].pret=98;  
strcpy(v[3].nume, "paine");
```

```
p = (struct Produs*) malloc (1 * sizeof(struct Elem));  
p->n=3;
```

Structuri cu câmpuri pe biți

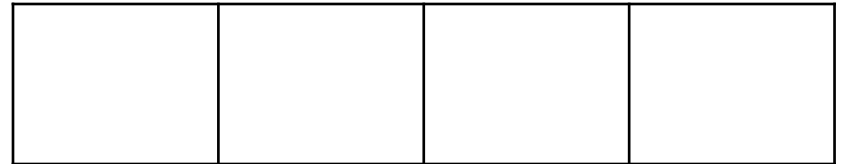
```
struct DataBiti {  
    unsigned int zi:5;  
    unsigned int luna:4;  
    int an:15;  
};
```

Dimensiune: 24 biți (se mărește la multiplu de 32 biți, 4 bytes, 1 int)



Uniuni

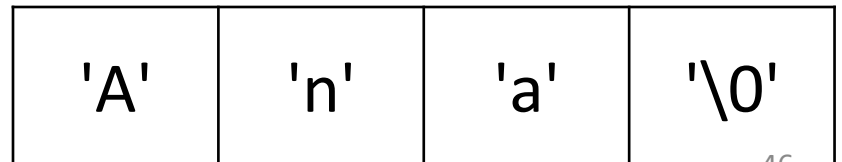
```
union int_sau_string {  
    int a;  
    char s[4];  
} x;
```



```
x.a=7;
```



```
strcpy(x.s, "Ana");
```



typedef la structuri

Structură normală

```
struct Data{  
    int zi;  
    int luna;  
    int an;  
};
```

```
struct Data d1,d2;  
int a = sizeof(struct Data);
```

Structură definită cu typedef

```
typedef struct {  
    int zi;  
    int luna;  
    int an;  
} Data;
```

```
Data d1,d2;  
int a = sizeof (Data);
```

Datele în programare

Un program gestionează date.

Datele pot fi:

- **date simple**: int, float, char etc.
- **structuri de date simple**: vector, matrice, struct, union etc.
- **structuri de date abstracte**: stiva, coada, dicționarul etc.

Stiva si Coada

Stiva este o structura de date LIFO (last in, first out), ceea ce inseamna ca ultimul element adaugat in stiva este primul element extras din stiva.

Coada este o structura de date FIFO (first in, first out), ceea ce inseamna ca primul element adaugat in coada o sa fie primul element extras din coada.

Stiva si Coada



Pointeri la funcții

Pointerii la funcții sunt variabile care stochează adresele funcțiilor în C.

În C, funcțiile sunt de fapt **adrese de memorie** care indică locația începutului funcției în memorie.

Pointerii la funcții sunt variabile care pot stoca aceste adrese și **pot fi utilizate pentru a apela funcții** sau pentru **a le trimite ca argumente către alte funcții**.

Pointeri la funcții

Pointerii la funcții sunt variabile care stochează adresele funcțiilor în C.

În C, funcțiile sunt de fapt **adrese de memorie** care indică locația începutului funcției în memorie.

Pointerii la funcții sunt variabile care pot stoca aceste adrese și **pot fi utilizate pentru a apela funcții** sau pentru **a le trimite ca argumente către alte funcții**.

Pointeri la functii – exemplu de utilizare

```
#include <stdio.h>
```

```
int f1(int a,int b)
{ return a+b; }
```

```
int f2(int a,int b)
{ return a-b; }
```

```
int main(void)
```

```
{ int a=7,b=5;
```

```
    int (*pf)(int,int);
```

```
    pf=&f1;
```

```
    printf("op(%d,%d)=>%d\n",a,b,(*pf)(a,b));
```

```
    pf=&f2;
```

```
    printf("op(%d,%d)=>%d\n",a,b,(*pf)(a,b));
```

```
    return 0;
```

```
}
```

```
op(7,5)=>12
op(7,5)=>2
```

// pf - pointer la o functie

// se seteaza pf cu adresa functiei f1

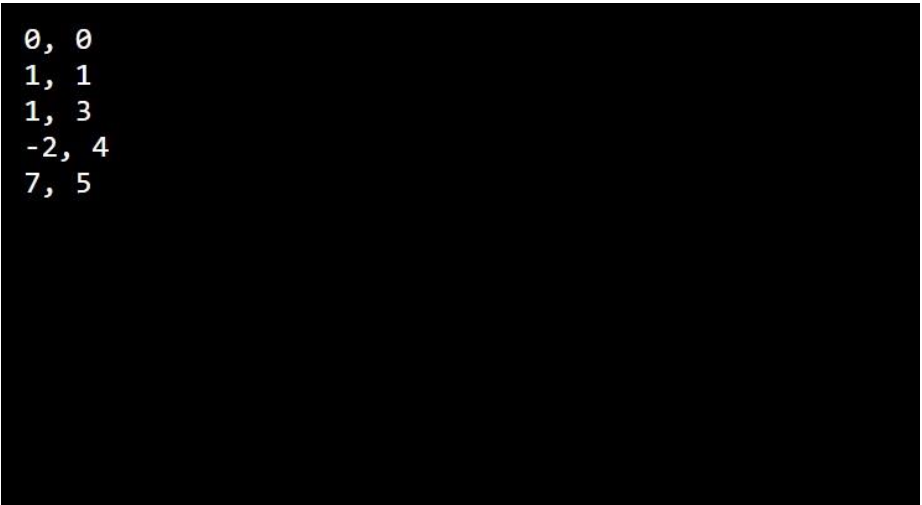
// se apelează functia pointata

qsort() – exemplu

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
```

```
typedef struct{
    double x,y;
}Pt;
```

```
int main(void)
{
    Pt puncte[5]={{1,3},{7,5},{0,0},{-2,4},{1,1}};
    int i,n=5;
    qsort(puncte,n,sizeof(Pt),cmpDist);
    for(i=0;i<n;i++){
        printf("%g, %g\n",puncte[i].x,puncte[i].y);
    }
    return 0;
}
```



```
0, 0
1, 1
1, 3
-2, 4
7, 5
```

qsort() – exemplu

```
double dist(const Pt *pt) // distanta fata de origine
{ return sqrt(pt->x*pt->x+pt->y*pt->y); }
```

// deoarece qsort transmite functiei de comparare adresele elementelor, functia va primi pointeri la elemente

// in acest caz, deoarece vectorul are elemente de tipul Pt, functia va primi parametri de tipul Pt*

```
int cmpDist(const void *elem1,const void *elem2)
```

```
{
    const Pt *p1=(const Pt*)elem1;
    const Pt *p2=(const Pt*)elem2;
    double d1=dist(p1);
    double d2=dist(p2);
    if(d1<d2) return -1;
    if(d1>d2) return 1;
    return 0;
}
```

```
0, 0
1, 1
1, 3
-2, 4
7, 5
```

bsearch() – exemplu utilizare

```
#include <stdio.h>
#include <stdlib.h>
```

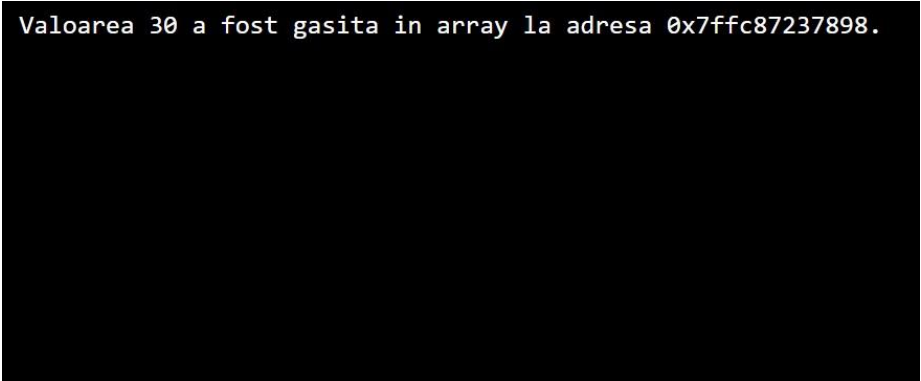
```
int cmpfunc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
```

```
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    int key = 30;
```

```
// Căutarea valorii 30 în array
```

```
int *ptr = (int*) bsearch(&key, arr, n, sizeof(int), cmpfunc);
```

```
if (ptr != NULL) {
    printf("Valoarea %d a fost gasita in array la adresa %p.\n", key, ptr);
} else {
    printf("Valoarea %d nu a fost gasita in array.\n", key);
}
return 0;
}
```



```
Valoarea 30 a fost gasita in array la adresa 0x7ffc87237898.
```


Matrici

int a[3][4];

*3 rânduri (de la 0 la 2),
4 coloane (de la 0 la 3),
12 elemente (3x4)*

	0	1	2	3
0				
1				
2				

Accesarea elementelor unei matrici

```
#define N 2  
#define M 3
```

```
int a[N][M];
```

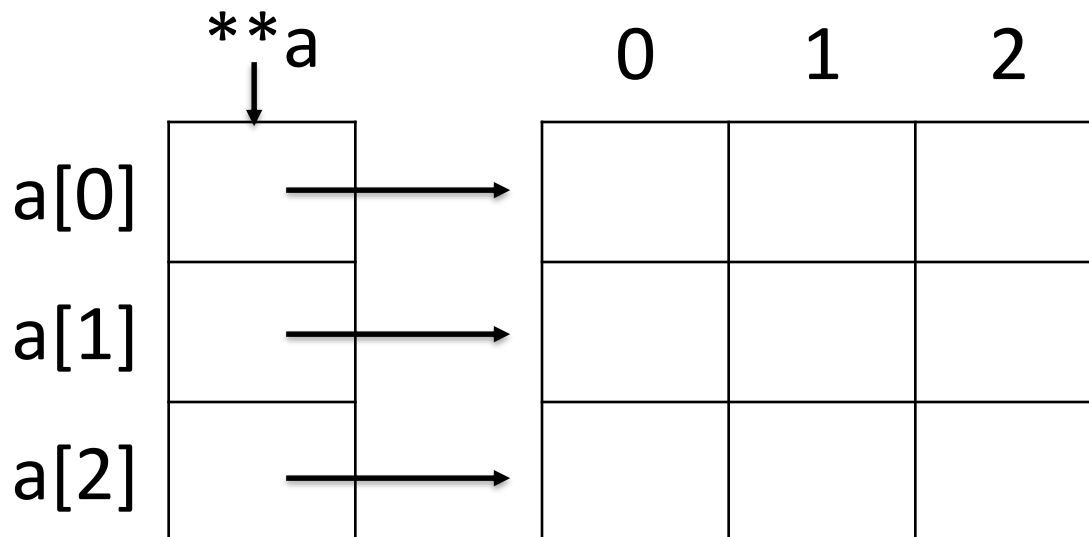
```
for(i=0; i<N;i++)  
    for(j=0; j<M; j++)  
    {  
        printf("a[%d][%d] =",  
            scanf("%d", &a[i][j]));  
    }
```

	0	1	2
0			
1			

Crearea unei matrici alocate dinamic

Matricea va fi definită ca `int **a`

`a[i]` este pointerul (`int*`) de la indexul `i` din `a`, adică adresa memoriei care conține acea linie



Crearea unei matrici alocate dinamic

```
int m,n,i,j;
```

```
int **a;
```

```
printf("m=");scanf("%d",&m);
```

```
printf("n=");scanf("%d",&n);
```

```
a=(int**)malloc(m*sizeof(int*));
```

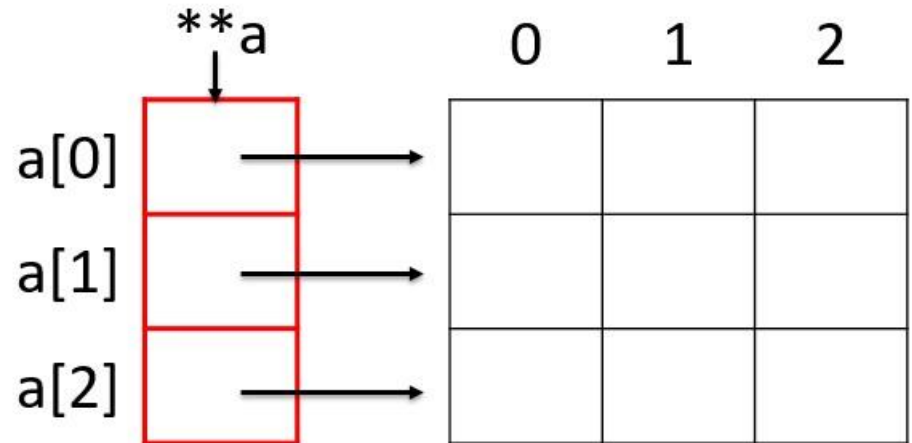
```
if(a==NULL)
```

```
{
```

```
    printf("memorie insuficienta\n");
```

```
    exit(EXIT_FAILURE);
```

```
}
```



Crearea unei matrici alocate dinamic

// alocare linii din matrice

```
for(i=0;i<m;i++)  
{
```

```
a[i]=(int*)malloc(n*sizeof(int));  
if(a[i]==NULL)
```

```
{
```

```
for(i--;i>=0;i--)
```

```
free(a[i]);
```

// elibereaza liniile alocate anterior

```
free(a);
```

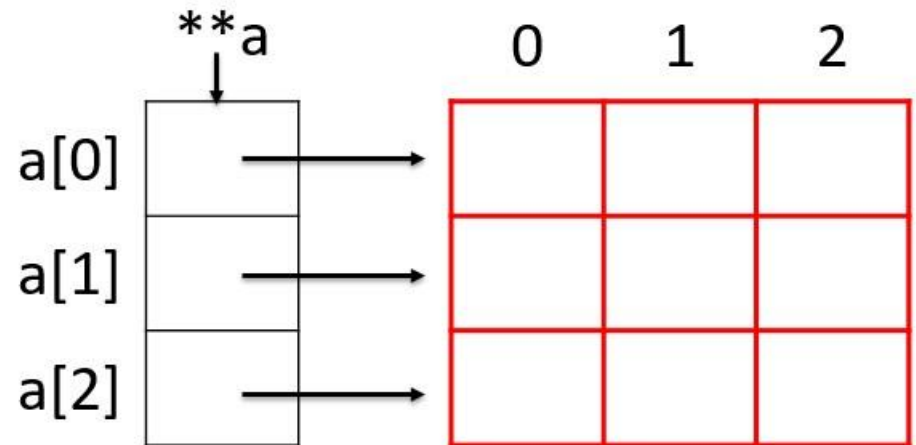
// elibereaza vectorul de pointeri

```
printf("memorie insuficienta\n");
```

```
exit(EXIT_FAILURE);
```

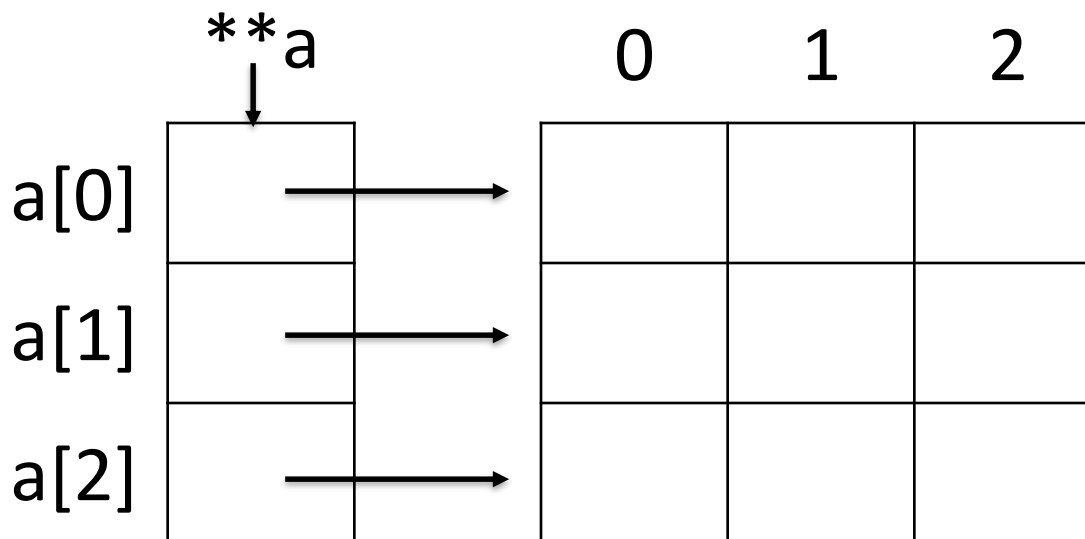
```
}
```

```
}
```



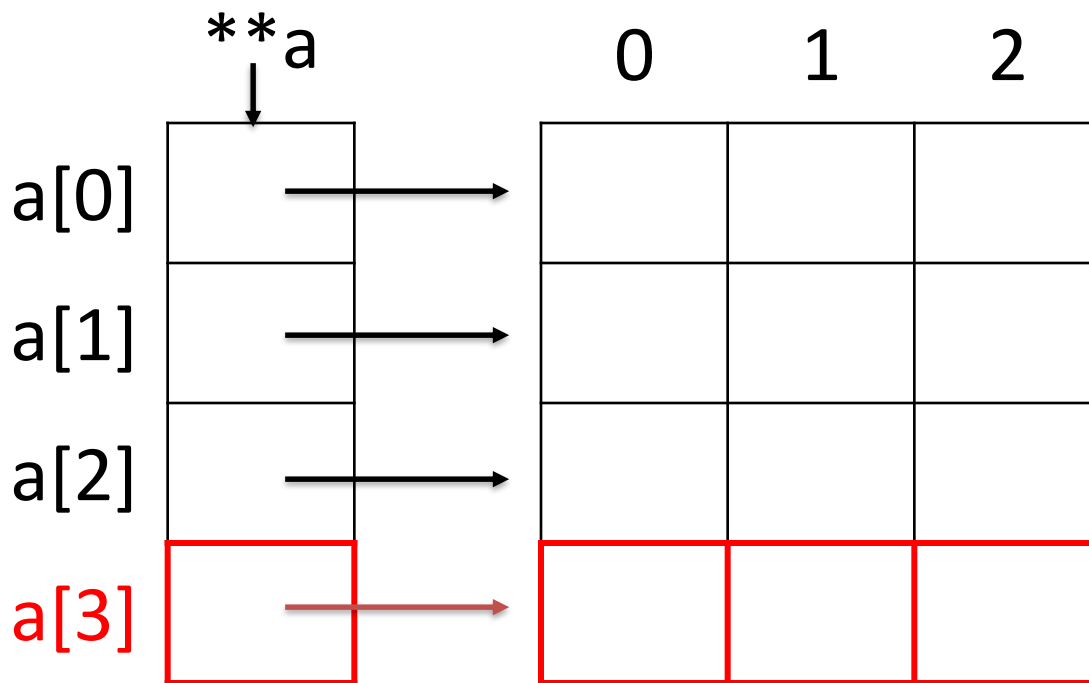
Modificarea dimensiunilor unei matrici alocate dinamic

Avantajul folosirii matricilor alocate dinamic (și a vectorilor alocați dinamic, în general) este că **putem redimensiona mărimea acestora**.



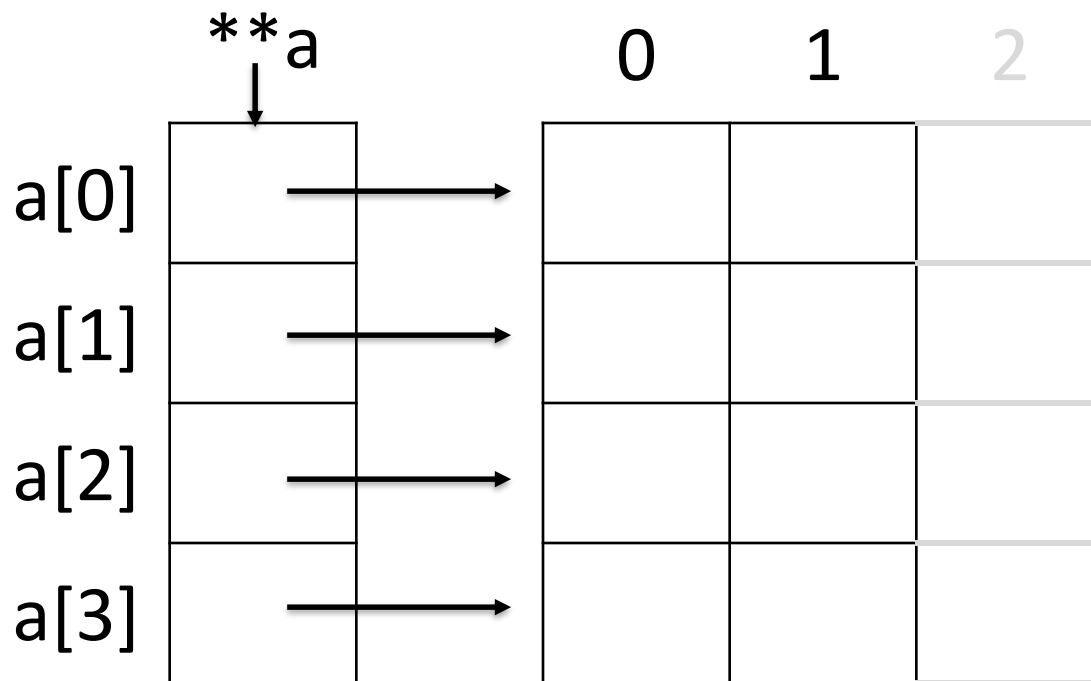
Modificarea dimensiunilor unei matrici alocate dinamic

Dacă vrem să adăugăm **o linie în plus** matricii va trebui să **redimensionăm vectorul de pointeri**, iar apoi **să alocăm dinamic memorie** pentru linia suplimentară.



Modificarea dimensiunilor unei matrici alocate dinamic

Dacă vrem să ștergem o coloană din matrice va trebui să redimensionăm fiecare linie din matrice (fiecare vector alocat dinamic).



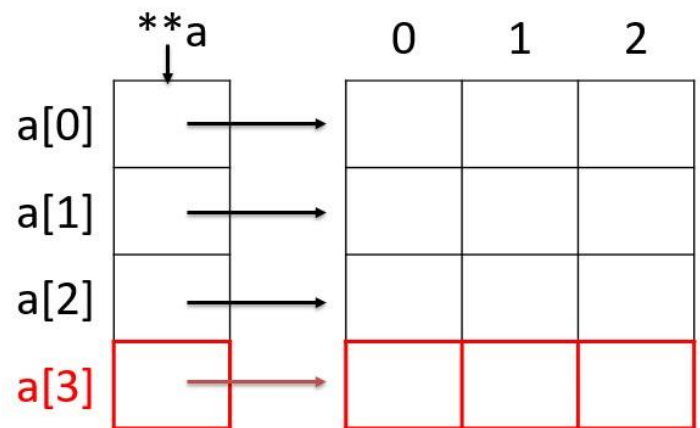
Modificarea dimensiunilor unei matrici alocate dinamic

Adăugarea unei linii suplimentare în matrice:

```
int ** adaugare_linie(int **a, int m, int n)
{
    int ** aux;
    aux = (int **) realloc(a,(m+1)*sizeof(int*));
    if(aux==NULL)
        free_mem(a,m,n);
    a=aux;
```

```
    a[m]= (int*)malloc(n*sizeof(int));
    if(a[m]==NULL)
        free_mem(a,m,n);

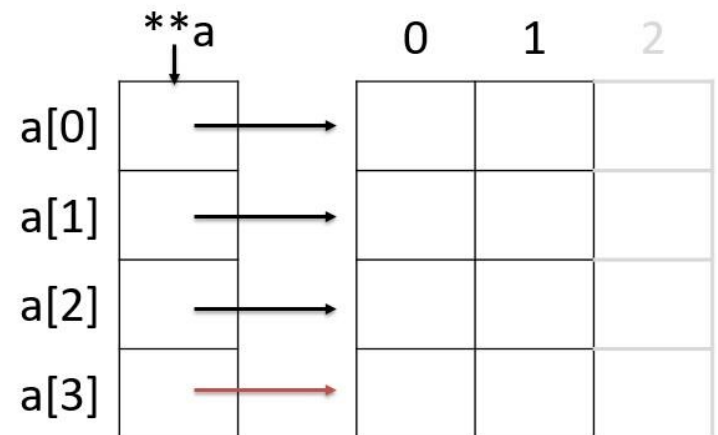
    return a;
}
```



Modificarea dimensiunilor unei matrici alocate dinamic

Ștergerea unei coloane din matrice:

```
void stergere_coloana(int **a, int m, int n)
{
    int *aux;
    for(int i=0;i<m;i++)
    {
        aux=(int*)realloc(a[i],(n-1)*sizeof(int));
        if(aux==NULL)
            free_mem(a,m,n);
        a[i]=aux;
    }
}
```



```
if (delete [ ] object)
{ delete [ ] object; }
else
{ if (currentSize != 0) delete [ ] object; }
endif

int size() const
{ return currentSize; }

Object & operator[] (int index)
{
    if (index < 0 || index >= currentSize)
        throw ArrayIndexOutOfBoundsException();
    return objects[index];
}

Object & operator[] (int index) const
{
    if (index < 0 || index >= currentSize)
        throw ArrayIndexOutOfBoundsException();
    return objects[index];
}
```

Vă mulțumesc!